Declaring a Router

A `modelId` is a promise that a name denotes a model. A router can keep it only by committing, before the request, to the set it may serve from.

The problem

`Bid` carries `provider`, `modelId`, `pricePerSecond` and three bookkeeping fields (`IBidStorage.sol`:14–21); none says what serves. A backend is a line of local JSON (`ai_engine.go`:58–64), and the requested model name is overwritten with the provider's configured one inside the adapter (`openai.go`:58). `providers/resale/overview.mdx` already tells a resale provider to bid on somebody else's model without the `tee` tag, since "you do not control the backend": candid, but not a protocol. Publishing the pool deletes routers, since pool and policy *are* the product; publishing nothing keeps the adverse selection of LP-315.

Commit to an admissible set

A provider registers a commitment $C = (root, floor, class, k)$ against its bid.

- **root**: a Merkle root over salted leaves $H(manifest_i \| s_i)$, one per servable model, a manifest fixing weights or upstream model, version, context length, quantisation; salt keeps an unopened leaf opaque.
- **floor**: a quality floor on a named public benchmark, claimed for every leaf.
- **class**: the class of selection rule, not the rule. Cheapest-above-floor, lowest-latency-above-floor and learned-score differ observably; parameters stay private.
- k: the size of the set. Public, because it bounds every privacy claim below.

Each response carries an opening: the leaf, its path, and a signature over chain id, diamond, `sessionId`, `requestId` and the response hash, which stops a flattering opening being replayed or moved between requests. Verification is one hash chain and one `secp256k1` recover, a few hundred bytes on a response already kilobytes wide.

What leaks, said plainly

Openings accumulate into the multiset of served leaves and their frequencies, the realised policy over the prompts a customer sent. k is the dial: $k = 4$ under a stable rule is public within a day, $k = 60$ stays a distribution, though hiding among sixty concedes the floor does little work. Estimating the policy to accuracy ε costs about k/ε^2 paid requests per prompt class, and re-salting each epoch restarts that bill.

Type the bid, not the model

Routing belongs to a provider's serving path, but `Model.tags` (`IModelStorage.sol`:22) is written by the model owner, and `isTeeModel` (`proxy_receiver.go`:123) already reads those tags to gate a check on a provider's backend. Put *fixed* against *routed* on the bid; buyers filter on a field, not a convention.

What each side gets

- **Routed providers** keep pool and policy, give up k, a floor and a rule class, and gain something to sell against competitors who disclose nothing.
- **Consumers** get a per-response proof that whatever answered sat inside a set fixed before the request, and a choice: filter routed out, or price it.
- **The network** keeps `modelId` meaning something, and admits routing deliberately.

First step

Nothing on chain has to move first. Whoever maintains `models-config.json` and the `aiengine` adapters can put C in the manifest at the model's `ipfsCID` and emit the opening as an added response field, the way `PongRes.Models` rides outside the signed payload so older consumers drop it and keep verifying (`request.go`:132–140). After a month of openings verified in the wild the root moves into a diamond slot. Hanzo runs the first router under this rule; we operate a learned router, so it constrains us most.

What it does not solve

It proves membership in a declared set: not that the set is good, not that the floor or the rule class held, which needs the challenge and stake layer of LP-313, and not that inference happened.